UCL SCHOOL OF MANAGEMENT

Module code: MSIN0097

Module title: Predictive Analytics

Assessment title: Group Coursework

Word count: 1988

Team 16

Name	Student Number
Chase Sun	25095830
Clara Yang	25149169
Leah Luo	25072251
Xiaocheng Wang	25070489

Table of Contents

1 Literature Review *3*

2 Practical exploration and benchmarking *5*

2.1 Benchmark design **5**

2.2 Task coverage and evaluation **5**

2.3 Results and findings **6**

3 Comparative analysis of agent tooling *7*

4 Reflection and conclusion *10*

References *11*

Appendix A: Benchmark Protocol (Task Specifications) *12*

Step 0: Setup **12**

Step 1: Dataset Ingestion + Schema Checks + Problem Definition **12**

Step 2: Exploratory Data Analysis **13**

Step 3: Missingness Handling **13**

Step 4: Baseline model training + evaluation harness **14**

Step 5: Improving performance **14**

Step 6: Producing Reproducible Packaging **15**

Step 7: Writing Documentation **15**

Appendix B: Benchmark success criteria *17*

Appendix C: Agent Run Logs *27*

C1 Claude Code **27**

C2 Codex **30**

C3 Antigravity **35**

Appendix D: Member contribution *37*

1 Literature Review

Agent Reasoning, Tool Use, and Knowledge Grounding

Reasoning and tool use are central to agent design, yet control strategies differ substantially. ReAct interleaves reasoning traces with actions to reduce hallucination (Yao et al., 2023), while Toolformer trains models to decide when API calls reduce predictive loss (Schick et al., 2023). RAG grounds generation in retrieved documents (Lewis et al., 2020), and DS-Agent applies this through case-based retrieval from Kaggle (Guo et al., 2024). Across these approaches, external grounding consistently improves factual accuracy, though whether control comes from prompting, learned tool use, or retrieval varies. The implications of these choices for reproducibility in real-world data science remain underexplored.

From Single-Agent Prompting to Multi-Agent Workflows

Multi-agent systems address end-to-end complexity through role specialisation. AutoGen enables flexible LLM–tool–human conversation (Wu et al., 2023), while MetaGPT encodes software-company SOPs into distinct agent roles, achieving 85.9% Pass@1 by requiring structured intermediate artefacts checkable by later workflow stages (Hong et al., 2023). Data Interpreter extends this to data science via hierarchical sub-problem decomposition and iterative code verification (Hong et al., 2024). Reflexion adds a self-reflection loop using verbal feedback, improving HumanEval Pass@1 from 80% to 91% (Shinn et al., 2023). A consistent finding is that verification checkpoints and structured outputs reduce cascading errors, though most evaluations still define success narrowly as task completion or metric improvement.

Benchmarks and Evaluation Gaps

Existing benchmarks reveal meaningful progress but leave important gaps. MLAgentBench reports a 37.5% success rate for a Claude v3 Opus agent across 13 tasks, dropping to 0% on post-cutoff challenges, indicating fragility on unfamiliar problems (Huang et al., 2024). SWE-bench reported initial resolution rates below 5% on 2,294 real GitHub issues (Jimenez et al., 2024), and MLE-bench found that even the strongest agent reached Kaggle bronze in only 16.9% of 75 competitions (Chan et al., 2024). DataSciBench similarly finds that models handle routine statistical tasks reasonably well but struggle on complex data mining and interpretability problems (Zhang et al., 2025).

Kapoor et al. (2024) argue these weaknesses partly reflect how agents are evaluated: benchmarks focus too narrowly on accuracy, conflate model performance with downstream usefulness, and suffer from weak holdout design and poor reproducibility. They also show that simple retry-based baselines can sometimes match more complex architectures at much lower cost. Overall, current benchmarks prioritise task completion over statistical validity — a significant limitation for data science, where runnable code may still produce unreliable results.

Challenges and Opportunities

Persistent challenges include hallucination in code generation, where models produce code unsuitable for the task or inconsistent with project context (Liu et al., 2024), and output variability sensitive to prompt wording (Sun et al., 2024; Wang et al., 2025). In data science settings, agents may introduce leakage, shift modelling choices across runs, or report performance gains without properly executing code (Huang et al., 2024). Results may also be inflated when benchmarks rely on public datasets that models have already seen.

The literature points to several directions for improvement: agents that check their own outputs during generation to catch leakage or schema mismatches early; selective human review when uncertainty or risk is high; and reproducibility embedded into the workflow itself, with agents producing seeds, dependency files, and run scripts alongside outputs.

Conclusion

Taken together, these studies suggest benchmark design should go beyond whether an agent produces runnable code. In data science, a pipeline can execute without errors and still be untrustworthy. Accordingly, the experiments in the following sections focus on the quality and reliability of the workflow, not merely whether it runs.

2 Practical exploration and benchmarking

2.1 Benchmark design

Three AI coding agents - **Claude Code**, **OpenAI Codex**, and **Antigravity** - were benchmarked against an identical end-to-end data science pipeline. All agents operated on the same dataset (UK Participation Survey 2024-25), the same binary classification target (CARTS_NET: arts engagement vs. non-engagement), predicted by the same set of features, and the same eight-step prompt protocol. No manual intervention was permitted except to recover from hard crashes. A run log records agent move.

The primary goal was not to improve predictive accuracy, but to assess end-to-end data science workflow capability of agents: could each agent ingest, clean, explore, model, and document a realistic ML task with methodological discipline? Key dimensions included workflow stability, methodological quality, reproducibility, and communication.

Experimental conditions are standardized to ensure a **fair comparison**[1]:

1. The dataset and tasks were fixed before any agent was run, so no agent received a task tailored to its strengths.
2. Prompts were delivered in a fixed sequence with no agent-specific wording.
3. Evaluation metrics were defined in the protocol, not chosen post-hoc.

In this way, differences in results were more likely to reflect differences in agent capability.

2.2 Task coverage and evaluation

The eight benchmark tasks were organized into four blocks to balance comparability with coverage of a realistic end-to-end data science workflow.

Block 1: Setup and problem framing

Included environment setup, logging structure, dataset ingestion, schema checks, and problem definition. Evaluation focused on whether agents could initialize a reproducible workflow correctly and define the task clearly. Key criteria covered file and notebook setup, schema-check completeness, problem framing, and variable description quality.

Block 2: Data preparation and exploratory analysis

Included target construction, EDA, and missingness handling. This block tested whether agents could inspect the data meaningfully and clean it without damaging downstream modelling. Key criteria covered task-relevant EDA coverage, appropriateness of visualizations, usefulness of

[1] Benchmark harness, prompt protocol, agent notebooks, and evidence artifacts are available at https://github.com/LanscorTK/predictive_team16

insights, and the quality and justification of missingness handling decisions.

Block 3: Modelling and performance improvement

Included baseline logistic regression, evaluation-harness design, hyperparameter tuning, XGBoost training, and final model selection. Here the benchmark assessed modelling discipline rather than raw accuracy alone. Key criteria covered split rigour, preprocessing quality, metric choice, tuning design, test-set discipline, fairness of comparison, and the quality of the final model-selection framework.

Block 4: Reproducibility and ducumentation

Included packaging and final reporting. This block tested whether agents could turn code into a reproducible and policy-facing deliverable. Key criteria covered the completeness of requirements.txt, clarity of README.md, consistency with notebook outputs, grounding of the final report in actual results, and suitability for a non-technical government audience.

Across all blocks, we used both **objective criteria** for task completion and **quality criteria** for implementation quality, to avoid treating runnable code as sufficient evidence of good data science practice.

2.3 Results and findings

Overall, all three agents completed the full eight-step workflow, but their strengths were distributed differently across tasks. The largest differences appeared not in final model choice, where all three converged on tuned logistic regression, but in intermediate decisions around data handling, exploratory depth, and evidence organization. Section 3 examines these differences through a structured comparative framework.

3 Comparative analysis of agent tooling

Completing the full workflow already represents a meaningful result given the survey's coded missing-value structure and the need to preserve routed responses. In overall terms, Claude Code and Codex were the more stable agents as they both handled cleaning conservatively, produced clearer evidence trails, and generated more inspectable outputs. Antigravity also completed the pipeline successfully, but its intermediate decisions were more brittle, particularly in data cleaning and documentation structure.

The clearest differences appeared in data preparation. Claude Code and Codex both retained high-frequency -3 routing codes as explicit categories where appropriate, especially for variables such as CULTSATIS, where 72% of respondents were routed past the question, whereas Antigravity removed -3 more broadly. As a result, Antigravity's cleaned dataset was around 4,000 rows smaller than the others, suggesting a more aggressive treatment of missingness. This was the stage at which agent behavior diverged most clearly, and it showed that benchmark differences were driven less by syntax or execution than by judgement about the data-generating process.

Table 1. Dataset sizes after each cleaning stage

Agent	Rows after EDA	Rows after cleaning	Drop rate (%)
Claude Code	34,338	29,073	15.3%
Codex	34,338	28,995	15.5%
Antigravity	34,338	24,867	27.6%

By contrast, modelling results were relatively similar. All three agents converged on tuned logistic regression as the final model, with test-set F2 scores between 0.452 and 0.488 and recall between 0.713 and 0.769. This suggests that once the pipeline was correctly specified, model choice was less important than the earlier decisions around cleaning and evaluation. The most consistent shared failure mode was the default 0.5 decision threshold where all baseline models initially missed almost all under-engaged respondents, and this was only corrected through validation-set threshold tuning using F2 and recall. EDA and documentation also revealed smaller differences, with Claude Code producing the deepest exploratory outputs and the most auditable evidence files.

Table 2. Final test-set performance of tuned logistic regression models

Agent	F2	Recall	Precision	ROC-AUC
Claude Code	0.452	0.769	0.170	0.829
Codex	0.464	0.713	0.194	0.831
Antigravity	0.488	0.735	0.208	0.853

A further parallel pattern was that tuned XGBoost did not outperform tuned logistic regression in the final model selection with all agents. This indicates that threshold optimization contributed more than additional model complexity in this imbalanced survey setting. The agents also differed in model-selection discipline. Codex used the most explicit multi-metric scoring formula, Claude Code applied a broader weighted framework including interpretability, and Antigravity relied on a simpler direct F2 comparison.

Table 3. Model selection framework

Agent	Approach	Weights	Selection logic
Claude Code	Weighted multi-criteria scoring	*Recall* 0.30, *F2* 0.25, *BalAcc* 0.20, *ROC-AUC* 0.15, *Interp* 0.10	Interpretability included explicitly
Codex	Policy-weighted score	*F2* 0.45, *Recall* 0.20, *PR-AUC* 0.15, *BalAcc* 0.10, *Prec* 0.10	Most formalized scoring rule
Antigravity	Direct F2 comparison	Direct *F2* comparison	Simplest decision rule

Strengths for tasks were differentiated. On exploratory analysis, Codex was the strongest overall, producing the richest and most policy-relevant visualizations, while Claude Code offered broader diagnostic coverage through correlation analysis and Antigravity remained more minimal but targeted.

Table 4. EDA performance

Agent	Plots generated	Depth	Unique contribution
Claude Code	***4 plots*** (distribution, features, correlations, feature-target)	Broad coverage + correlation heatmap	Correlation analysis
Codex	***6 plots*** (target, age, education, deprivation, wellbeing KDE, CULTSATIS)	Policy-focused + KDE plots	Wellbeing KDE by engagement
Antigravity	***3 plots*** (age, education, deprivation)	Minimal but targeted	Deprivation decile analysis

Reproducibility was broadly strong across all three agents, with fixed seeds and pinned dependencies, but the audit trail was clearest in Claude Code and Codex, whereas Antigravity's programmatic notebook assembly made intermediate steps less transparent to inspect.

Search effort also differed substantially. Claude Code was the most exhaustive in

hyperparameter exploration, whereas Antigravity reached the strongest final F2 with a much simpler tuning process. In engineering terms, Claude Code produced the cleanest execution and most thorough tuning artefacts, Codex the richest structured evidence trail, and Antigravity a more compact but less inspectable workflow. No major safety or compliance issues were observed in these runs.

Table 5. Engineering execution and evidence outputs by agent

Agent	Evidence Trail	LR tuning configs	XGB tuning configs	Execution issues
Claude Code	**2** CSVs + **4** plots	**153** (9C × 17 thresholds)	**459** (27 × 17 thresholds)	None
Codex	**2** CSVs + **2** JSONs + **6** plots + clean data	**28** (7C × 2pen × 2wt)	**12** candidates	Initial indentation fix
Antigravity	**3** plots + model pickle	**8** (4C × 2wt)	**12** candidates	XGB deprecation warning

Table 6 provides a compact summary of performance of the three tests agents across 10 criteria, based on the success criteria of every step shown in Appendix B, where both the basic functionality and quality are assessed.

Table 6. Summary comparison of agent performance across benchmark criteria

Criterion	Claude Code	Codex	Antigravity
Data handling	++	++	+
EDA depth	++	+++	+
Test recall	+++	++	++
Test F2	++	++	+++
Test ROC-AUC	++	++	+++
Model selection rigor	++	+++	+
Tuning thoroughness	+++	++	+
Evidence trail	++	+++	+
Code quality	+++	+++	++
Report quality	++	++	++

Note: + relatively weaker, ++ moderate, +++ relatively stronger within this benchmark

4 Reflection and conclusion

The literature identifies verification gaps, reproducibility challenges, and code hallucination as persistent risks when LLM-based agents are applied to data science (Kapoor et al., 2024; Liu et al., 2024). Our benchmark confirms these concerns. All three agents produced runnable pipelines, yet they diverged most on decisions requiring domain judgement, especially in the interpretation of survey routing codes and choice of metrics for imbalanced data. Task completion alone proved an insufficient indicator of agent quality.

Several practical lessons emerged from the experiments. Threshold tuning was the single most impactful intervention by raising minority class recall from under 6% to over 70%, with larger effects than model switching or feature engineering. Missingness handling had the greatest influence on data quality, while structured evidence trails made audit feasible where inline notebook outputs did not. Across both the literature and our runs, the most reliable pattern was not full automation, but agent use within a clearly verified workflow.

Based on these findings, we propose a short playbook for future teams.

1. **Recommended workflow patterns:** fix prompts, seeds, package versions, and evaluation metrics before running any agent; require each run to produce inspectable artefacts such as logs, plots, saved metrics, and intermediate outputs.
2. **Verification checklist:** after each run, confirm class balance assumptions, missingness handling, split discipline, leakage controls, threshold choice, and whether the written documentation matches the actual outputs.
3. **Common failure modes:** default 0.5 thresholds on imbalanced targets, overly narrow tuning grids, silent row dropping, documentation-to-output mismatch, and incorrect interpretation of coded variables.
4. **When not to use agent tooling:** agent workflows are less suitable when datasets depend on domain-specific coding schemes, survey routing logic, or other contextual conventions that cannot be safely inferred from metadata alone, especially when there is limited time for independent validation.

References

Chan, J.S. et al. (2024) 'MLE-bench: Evaluating machine learning agents on machine learning engineering', arXiv preprint. Available at: https://arxiv.org/abs/2410.07095.

Guo, S. et al. (2024) 'DS-Agent: Automated data science by empowering large language models with case-based reasoning', in Proceedings of the 41st International Conference on Machine Learning (ICML). PMLR 235, pp. 16813–16848.

Hong, S. et al. (2023) 'MetaGPT: Meta programming for a multi-agent collaborative framework', in The Twelfth International Conference on Learning Representations (ICLR 2024).

Hong, S. et al. (2024) 'Data Interpreter: An LLM agent for data science', in Findings of the Association for Computational Linguistics: ACL 2025, pp. 19796–19821.

Huang, Q. et al. (2024) 'MLAgentBench: Evaluating language agents on machine learning experimentation', in Proceedings of the 41st International Conference on Machine Learning (ICML).

Jimenez, C.E. et al. (2024) 'SWE-bench: Can language models resolve real-world GitHub issues?', in The Twelfth International Conference on Learning Representations (ICLR 2024).

Kapoor, S. et al. (2024) 'AI agents that matter', Transactions on Machine Learning Research. Available at: https://arxiv.org/abs/2407.01502.

Lewis, P. et al. (2020) 'Retrieval-augmented generation for knowledge-intensive NLP tasks', in Advances in Neural Information Processing Systems, 33, pp. 9459–9474.

Liu, F. et al. (2024) 'LLM hallucinations in practical code generation: Phenomena, mechanism, and mitigation', Proceedings of the ACM on Software Engineering, 2(ISSTA).

Schick, T. et al. (2023) 'Toolformer: Language models can teach themselves to use tools', in Advances in Neural Information Processing Systems (NeurIPS 2023).

Shinn, N. et al. (2023) 'Reflexion: Language agents with verbal reinforcement learning', in Advances in Neural Information Processing Systems (NeurIPS 2023).

Sun, M. et al. (2024) 'A survey on large language model-based agents for statistics and data science', The American Statistician. doi: 10.1080/00031305.2025.2561140.

Wang, P. et al. (2025) 'Large language model-based data science agent: A survey', arXiv preprint. Available at: https://arxiv.org/abs/2508.02744.

Wu, Q. et al. (2023) 'AutoGen: Enabling next-gen LLM applications via multi-agent conversation', arXiv preprint. Available at: https://arxiv.org/abs/2308.08155.

Yao, S. et al. (2023) 'ReAct: Synergizing reasoning and acting in language models', in The Eleventh International Conference on Learning Representations (ICLR 2023).

Zhang, D. et al. (2025) 'DataSciBench: An LLM agent benchmark for data science', arXiv preprint. Available at: https://arxiv.org/abs/2502.13897.

Appendix A: Benchmark Protocol (Task Specifications)

The following 8 step - 14 prompts were delivered to all three agents in fixed order. Each agent received identical instructions with only the agent name substituted.

Step 0: Setup

Work in ./agents/[agent_name] and NOWHERE ELSE. Assume the following files are available in ./data:

1. Variable dictionary: participation_2024-25_data_dictionary_v2.txt
2. Data file: participation_2024-25_experiment_v2.tab

Read the variable dictionary to understand the meanings of the variables and their coded values (just try to understand, no need creating note in the notebook). Do not load the data file yet in this step.

Create a Jupyter Notebook named `experiment_[agent_name].ipynb`.

Add a markdown cell at the top of the notebook containing the main title: # Experiment: [agent_name] and the name of this section, with correct level.

Set the global random seed to 42. Use `random_state=42` in all later steps whenever randomness is involved, to ensure reproducibility.

Use only relative paths. Read all inputs from the current directory and save all outputs to the current directory. Ensure the notebook can run sequentially from top to bottom without any manual intervention.

Also set up the following experiment-tracking structure in the current directory:

- a run log file named `run_log_[agent_name].md`
- an evidence folder named `evidence_[agent_name]`

Use the run log to record progress step by step throughout the experiment. For each major step, record:

- step name
- completion status
- key actions
- key outputs
- important warnings or errors, if any

Save all major output files generated during the experiment in the evidence folder, unless a later step specifies a different required location.

If any failure prevents reliable continuation of the pipeline, mark it clearly in the run log as `EARLY FAILURE`, and state:

- which step failed
- the reason for failure
- which later steps were affected

Step 1: Dataset Ingestion + Schema Checks + Problem Definition

Step 1.1: Dataset Ingestion and Schema Checks

Read the data file `participation_2024-25_experiment_v2.tab` into a pandas dataframe called participation_raw, and report its number of rows and columns.

The dataset is expected to contain the following variables:

`CARTS_NET, CHERVIS12_NET, EDUCAT3, NSSEC_3, FINHARD, WELLB1, CINTOFT, CSMARTD_Count, WELLB4, TENHARM, LONELY, ETHNIC_NET, emdidc19, CULTSATIS, SEX, AGEBAND`

Perform appropriate schema checks for these variables and document in a markdown cell what checks were performed. At minimum, confirm that the required variables are present and check for any obvious schema

issues relevant to this step.

Create a markdown cell for this subsection that includes:

- a level 1 heading
- a level 2 heading
- a short description of the work completed in this subsection

Then create a code cell that implements this step.

Step 1.2: Problem Definition

Create a markdown cell with a level 2 heading defining the prediction task.

The task is a binary classification task: predict whether a respondent engaged with the arts physically in the last 12 months.

We frame this task as an under-engagement identification problem with social research value. Rather than treating arts participation as a purely individual preference, the task investigates whether non-participation is socially patterned across demographic, socioeconomic, digital, and geographic factors. The purpose is to identify groups that may face structural or contextual barriers to physical arts engagement, and to provide evidence for more inclusive cultural policy and public engagement strategies.

Define:

- target variable: `CARTS_NET`
- feature variables: `CHERVIS12_NET, EDUCAT3, NSSEC_3, FINHARD, WELLB1, CINTOFT, CSMARTD_Count, WELLB4, TENHARM, LONELY, ETHNIC_NET, emdidc19, CULTSATIS, SEX, AGEBAND`

Also state clearly in the markdown cell that for the target variable, rows with values `-3` and `3` will later be dropped as missing values so that the task becomes a binary classification problem. Do not drop them yet in this step.

In the same markdown cell, include a table introducing all variables, based on the variable dictionary file `participation_2024-25_data_dictionary_v2.txt`.

Step 2: Exploratory Data Analysis

First, convert the target into a binary classification target for later use. Remove all rows where `CARTS_NET` takes values `-3` or `3`, as these are treated as missing target values. Do not recode the remaining target values to `0` and `1` yet.

Create a copy of the data for subsequent steps: drop the original `CARTS_NET` column and add a new binary target variable to this new dataframe called participation_eda.

Create a new folder named `EDA_[agent_name]_Pics` in the evidence folder created in section 0. Save all figures generated in this step as `.png` files in that folder.

Perform appropriate exploratory data analysis for the prediction task, based on the variable definitions and roles. The EDA should generate useful data insights and must include suitable visualisations using appropriate graph types.

Organise this section clearly in the notebook using markdown and code cells. Include correct heading levels and add subheadings where helpful. Follow a readable flow in which markdown cells explain the work and code cells implement it.

Do not perform any modelling, training, or model evaluation in this step.

Step 3: Missingness Handling

In this dataset, missing values are not stored as `NaN`, but are often represented by negative codes. For this step, focus only on the feature variables, since the target variable has already been handled earlier. For variables

with coded missing rates under 5%, consider dropping those rows. For CULTSATIS which has ~71% routing skips, treat -3 as its own informative category rather than missing.

Using the variable dictionary `participation_2024-25_data_dictionary_v2.txt`, review the meanings of the coded values for each feature variable and determine how to handle values that do not carry useful information for modelling, such as missing, not applicable, or similar non-informative responses.

You may need to define variable-specific missingness handling rules, depending on the missing rate and meanings of different coded values. Clearly explain these handling rules in markdown.

Organise this section clearly in the notebook using appropriate heading levels. Follow a readable flow in which markdown cells explain the work and code cells implement it.

Print necessary information, such as rows before and after cleaning.

Conduct the missingness handling job on the data frame "participation_eda" created in the last section, and the expected output is a cleaned dataset "participation_clean"with no missing value, ready for encoding and training.

Step 4: Baseline model training + evaluation harness

Step 4.1: Prepare Modelling Data

Organise this section 4 clearly in the notebook using appropriate heading levels. Follow a readable flow in which markdown cells explain the work and code cells implement it.

Using the cleaned dataframe "participation_clean" from the previous steps, define `X` and `y`.

Create appropriate preprocessing pipelines for Logistic Regression and XGBoost respectively, including any necessary preprocessing such as one-hot encoding.

Split the data into training, validation, and test sets in a 0.7 / 0.15 / 0.15 ratio. Account for target imbalance when performing the split.

Step 4.2: Create Evaluation Harness

Define a common set of evaluation metrics and outputs suitable for both Logistic Regression and XGBoost, so that model performance can be assessed and compared consistently.

When designing, consider the real-world context of the task and the characteristics of the data to determine the metrics.

Explain the rules for evaluation harness and the design rationale in the markdown cell.

Step 4.3: Baseline Model (Logistic Regression)

Train a baseline Logistic Regression model. Choose reasonable baseline hyperparameters based on standard practice, without extensive tuning.

Evaluate the model on the validation set only, using the predefined evaluation harness and present the results clearly.

Step 5: Improving performance

Step 5.1: Improve Logistic Regression

Organise this section clearly in the notebook using appropriate heading levels. Follow a readable flow in which markdown cells explain the work and code cells implement it.

Use exactly the same training, validation, and test split defined earlier.

Tune the Logistic Regression model on the validation set only and identify the best-performing hyperparameter setting including threshold tuning. Consider using class_weight='balanced' or scale_pos_weight for handling imbalance, and explore threshold tuning systematically (e.g., optimise the threshold for F2 score on the validation set).

Step 5.2: Tune XGBoost

Use exactly the same training, validation, and test split defined earlier.

Train an XGBoost model and tune it on the validation set only to identify the best-performing hyperparameter setting including threshold tuning. Consider using class_weight='balanced' or scale_pos_weight for handling imbalance, and explore threshold tuning systematically (e.g., optimise the threshold for F2 score on the validation set).

Step 5.3: Model Comparison

Only at this stage, use the test set for prediction and final comparison.

Compare the following models:

- baseline Logistic Regression
- tuned Logistic Regression
- XGBoost

Use the evaluation harness defined in the previous section to assess and compare model performance consistently.

Step 5.4: Final Model Decision

Taking into account the purpose of the prediction task, define a systematic, multi-dimensional, and quantitative model selection framework in a markdown cell.

Clearly explain the selection criteria, then use this framework to choose the final model.

For both tuned Logistic Regression and tuned XGBoost, print a structured tuning summary in the notebook. For each model, report:

- tuning method used
- hyperparameters searched
- search range or candidate values for each hyperparameter
- total number of parameter configurations evaluated
- iteration or trial count completed
- best hyperparameter setting found
- best validation-set performance achieved under the predefined evaluation harness

Step 6: Producing Reproducible Packaging

Do not add any new content to the notebook in this step.

In the working directory, create a `requirements.txt` file that includes all Python packages used in the experiment.

Also create a short `README.md` file that clearly states:

- which input files are required
- how to run the notebook
- what outputs are produced and where they are saved
- what steps were taken to support reproducibility, such as fixed random seed control

Keep the packaging concise, clear, and consistent with the workflow implemented in the notebook.

Step 7: Writing Documentation

Do not add any new content to the notebook in this step.

Based on the completed notebook results, write a non-technical English report for a government arts department and save it in the workspace as `Report_[agent_name].md`.

The report should be approximately 400 words and written in markdown format.

The report should follow a clear high-level structure, but you may organise the wording and emphasis as you judge appropriate:

- the purpose of the analysis
- a brief overview of the data and approach
- the main findings from the modelling results
- the final model choice and why it was selected
- the practical implications for public arts engagement
- key limitations and cautions

Write for a policy-facing, non-technical audience. Keep the language clear and accessible. Avoid code, formulas, and unnecessary technical jargon. Do not overclaim what the model can do, and do not imply causal conclusions from predictive results alone.

Use the actual results from the notebook rather than generic placeholder language.

Appendix B: Benchmark success criteria

Step 0: Setup

Objective criteria (basic function)

1. The notebook is created with the correct filename: experiment_[agent_name].ipynb
2. The first cell is a markdown cell containing the main title # Experiment: [agent_name]
3. The notebook includes the section title for Step 0 with an appropriate heading level
4. The two required files are acknowledged in this step:
5. The variable dictionary is consulted in this step, but the data file is not loaded
6. A fixed seed of 42 is defined for later use
7. Only relative paths are used
8. The notebook is structured to run sequentially from top to bottom without manual intervention
9. A run log file named run_log_[agent_name].md is created
10. An evidence folder named evidence_[agent_name] is created
11. The run log includes the required fields for step tracking
12. The run log specifies how EARLY FAILURE should be recorded

Quality criteria

1. Setup clarity and completeness

What is being judged: whether the setup is clear, complete, and usable for the rest of the pipeline.

- 5 = Setup is complete, clear, and directly supports later steps with no ambiguity
- 4 = Setup is mostly complete and clear, with only minor omissions or awkwardness
- 3 = Setup is usable but somewhat incomplete, vague, or inconsistent
- 2 = Setup is only partially usable and leaves important points unclear
- 1 = Setup is confusing, incomplete, or not usable as a reliable starting point

2. Reproducibility readiness

What is being judged: whether the setup establishes strong foundations for reproducible execution.

- 5 = Reproducibility measures are clearly and correctly established, including seed control, path discipline, and sequential execution readiness
- 4 = Reproducibility is mostly well established, with only minor weaknesses
- 3 = Some reproducibility measures are present, but important details are weak or incomplete
- 2 = Reproducibility is addressed superficially and would likely cause issues later
- 1 = Reproducibility is poorly addressed or effectively absent

3. Logging and evidence setup quality

What is being judged: whether the run log and evidence structure are well prepared for tracking and auditing the experiment.

- 5 = Logging and evidence setup are clear, well-structured, and practical for later monitoring and comparison
- 4 = Logging and evidence setup are solid, with only minor limitations
- 3 = Basic logging and evidence structures exist, but they are not fully clear or robust
- 2 = Logging and evidence structures are weak, incomplete, or difficult to use
- 1 = Logging and evidence setup are missing, unclear, or unusable

Step 1: Dataset Ingestion + Schema Checks + Problem Definition

Objective criteria (basic function)

1. The data file is read successfully into a pandas dataframe
2. The notebook reports the dataset shape correctly: number of rows and columns

3. All required variables are checked and confirmed to exist, or any missing variables are clearly reported
4. Appropriate schema checks are performed for the specified variables and the checks are documented in a markdown cell
5. A markdown cell is created for Section 1.1 with the required level 1 heading, level 2 heading, and brief description
6. A code cell is created that implements dataset ingestion and schema checks
7. A markdown cell is created for Section 1.2 with a clear definition of the binary classification task
8. The target variable and all ten feature variables are stated correctly
9. The markdown clearly notes that rows with CARTS_NET values -3 and 3 will be dropped later, but are not dropped in this step
10. A variable summary table is included and matches the variable dictionary file

Quality criteria

1. Quality of schema checks

What is being judged: whether the schema checks are sufficiently comprehensive, correct, and relevant to this prediction task.

- 5 = Checks go well beyond basic variable presence. They cover dataset shape, required variable presence, and at least two of the following: if there is duplicate column, observed values, coding consistency, or obvious structural issues. The checks are clearly reported and useful for later modelling.
- 4 = Checks include dataset shape, required variables, and at least one additional useful check, but miss one important aspect.
- 3 = Checks cover the basics only: dataset shape, required variable presence, and one limited additional check. Coverage is adequate but narrow.
- 2 = Checks are minimal: only dataset shape and required variable presence are confirmed, with little or no further inspection.
- 1 = Checks are missing, very unclear, or incorrect.

2. Quality of problem definition

What is being judged: whether the prediction task is defined accurately, clearly, and with meaningful substantive context.

- 5 = The task is defined clearly and accurately, including the prediction goal, target, features, later target cleaning rule, and a clear statement of the task's real-world or policy relevance.
- 4 = The task definition is clear and mostly complete, with some meaningful substantive context, but the practical significance is not fully developed.
- 3 = The task definition is broadly correct, but somewhat generic, incomplete, or weakly connected to the task's real-world significance.
- 2 = The task definition is partly correct but vague, with little meaningful context or practical motivation.
- 1 = The task definition is unclear, inaccurate, or poorly aligned with the task.

3. Quality of variable description table

What is being judged: whether the variable table is useful, accurate, and grounded in the variable dictionary.

- 5 = The table is complete, accurate, and clearly based on the variable dictionary. It is easy to use for later analysis.
- 4 = The table is mostly accurate and complete, with only minor omissions or wording issues.
- 3 = The table is usable but somewhat incomplete, uneven, or imprecise.
- 2 = The table has noticeable omissions, inaccuracies, or limited usefulness.
- 1 = The table is missing or seriously inaccurate.

4. Quality of notebook communication and structure

What is being judged: whether the section is well organised and easy to follow as part of a reproducible analysis notebook.

- 5 = Markdown and code are very well structured, with clear headings, logical flow, and concise explanations.
- 4 = The section is clear and readable overall, with only minor issues in flow or organisation.
- 3 = The section is understandable but somewhat uneven in structure or explanation.
- 2 = The section is hard to follow in places because of weak organisation or unclear markdown.
- 1 = The section is poorly organised or confusing.

Step 2: EDA and insight generation

Objective criteria (basic function)

1. Rows with CARTS_NET values -3 and 3 are removed correctly
2. A new dataframe is created for subsequent steps
3. The original CARTS_NET column is dropped from the new dataframe
4. A new binary target variable is added to the new dataframe
5. The folder EDA_[model_name]_Pics is created in the current directory
6. All EDA figures are saved as .png files in that folder
7. The EDA is relevant to the prediction task and includes appropriate visualisations
8. Markdown and code cells are used in a clear, readable sequence
9. Headings and subheadings are structured correctly
10. No modelling, training, or model evaluation is performed in this step

Quality criteria

1. Quality of EDA coverage and task relevance

What is being judged: whether the EDA is sufficiently relevant to the prediction task and covers the most important aspects of the data.

- 5 = The EDA is clearly focused on the prediction task and covers the key elements needed for later modelling, including target distribution, feature distributions, and feature-target relationships.
- 4 = The EDA is mostly relevant and covers several important aspects, but misses one useful area of analysis.
- 3 = The EDA is broadly relevant but somewhat limited, uneven, or too descriptive.
- 2 = The EDA has weak focus on the prediction task and covers only a narrow or superficial set of analyses.
- 1 = The EDA is poorly targeted, largely uninformative, or not meaningfully connected to the task.

2. Quality of visualisation choices

What is being judged: whether the visualisations are appropriate for the variables, readable, and useful for interpretation.

- 5 = Visualisations are well chosen for the data types and task, clearly labelled, readable, and genuinely helpful for interpretation.
- 4 = Visualisations are mostly appropriate and readable, with only minor weaknesses in choice or presentation.
- 3 = Visualisations are usable but somewhat repetitive, suboptimal, or only moderately informative.
- 2 = Visualisations are poorly chosen for the variables, hard to read, or add limited analytical value.
- 1 = Visualisations are missing, seriously inappropriate, or very difficult to interpret.

3. Quality of insights and modelling awareness

What is being judged: whether the EDA produces meaningful insights and identifies issues relevant to later modelling.

- 5 = Insights are accurate, useful, and well connected to the modelling task. The section identifies important risks or considerations such as class imbalance, sparse categories, skewed distributions, or other issues likely to affect preprocessing or model performance.

- 4 = Insights are mostly useful and accurate, and identify at least one important modelling consideration, but do not develop all relevant implications.
- 3 = Insights are broadly reasonable but fairly descriptive, with limited attention to modelling implications.
- 2 = Insights are weak, generic, or only loosely supported by the analysis, with little awareness of later modelling risks.
- 1 = Insights are missing, misleading, or not supported by the EDA.

4. Quality of notebook communication and structure

What is being judged: whether the section is well organised and easy to follow as part of a reproducible analysis workflow.

- 5 = Markdown and code are very well structured, with clear headings, logical flow, and concise explanations that support interpretation.
- 4 = The section is clear and readable overall, with only minor issues in flow or organisation.
- 3 = The section is understandable but somewhat uneven in structure or explanation.
- 2 = The section is hard to follow in places because of weak organisation or unclear markdown.
- 1 = The section is poorly organised or confusing.

Step 3: Missingness handling

Objective criteria (basic function)

1. Missingness handling is performed on participation_eda
2. Missing-value rules are defined using the variable dictionary
3. Rules are explained clearly in markdown
4. Variable-specific handling is applied where needed
5. Rows before and after cleaning are reported
6. The cleaned output is saved as participation_clean
7. participation_clean contains no missing values in the feature variables
8. The cleaned dataset is ready for encoding and model training
9. Markdown and code cells are organised clearly with appropriate headings

Quality criteria

1. Quality of missingness handling strategy

What is being judged: whether missing and non-informative coded values are handled appropriately for the prediction task, without unnecessary data loss or obviously poor decisions.

- 5 = Handling is thoughtful and variable-specific. The approach considers both code meaning and missingness rate, avoids unnecessary row dropping, and makes sensible distinctions between dropping, recoding as missing, or retaining values where appropriate.
- 4 = Handling is generally sound and mostly variable-specific, with only minor weaknesses such as one slightly over-simplified or weakly justified decision.
- 3 = Handling is broadly workable but somewhat blunt. Some variable-specific reasoning is present, but the approach is limited, inconsistent, or causes avoidable data loss.
- 2 = Handling is simplistic or poorly suited to the task, for example relying heavily on row deletion or using the same treatment for most variables without enough justification.
- 1 = Handling is clearly poor, causing major unnecessary data loss, obvious bias risk, or serious misuse of coded values.

2. Quality of rule definition and justification

What is being judged: whether the missingness rules are clearly defined, interpretable, and grounded in the variable dictionary.

- 5 = Rules are clear, well organised, and explicitly linked to the coded meanings in the variable dictionary. The rationale is easy to follow.
- 4 = Rules are mostly clear and reasonably justified, with only minor gaps in explanation or organisation.
- 3 = Rules are present and partly justified, but some decisions are vague, weakly explained, or not clearly tied to the dictionary.
- 2 = Rules are incomplete, weakly justified, or difficult to follow.
- 1 = Rules are missing, very unclear, or not meaningfully justified.

3. Quality of cleaned output for downstream modelling

What is being judged: whether participation_clean appears well prepared for later encoding and model training.

- 5 = participation_clean is clearly usable for downstream modelling, with no unresolved feature missingness and no obvious cleaning inconsistencies.
- 4 = participation_clean is mostly ready for modelling, with only minor issues or weak spots.
- 3 = participation_clean is usable but somewhat uneven, with minor remaining concerns about consistency or preparation quality.
- 2 = participation_clean has noticeable weaknesses that may create problems in later encoding or modelling.
- 1 = participation_clean is not reliably ready for downstream use.

4. Quality of notebook communication and structure

What is being judged: whether the section is well organised and easy to audit as part of a reproducible cleaning workflow.

- 5 = Markdown and code are very well structured, with clear headings, logical flow, and useful reporting of before/after cleaning results.
- 4 = The section is clear and readable overall, with only minor issues in flow or explanation.
- 3 = The section is understandable but somewhat uneven in organisation or reporting.
- 2 = The section is hard to follow in places because of weak structure or unclear explanations.
- 1 = The section is poorly organised or confusing.

Step 4: Baseline model training + evaluation harness

Objective criteria (basic function)

1. X and y are defined correctly from participation_clean
2. Preprocessing pipelines are created for both Logistic Regression and XGBoost
3. The data is split into training, validation, and test sets in a 0.7 / 0.15 / 0.15 ratio
4. The split accounts for target imbalance
5. A common evaluation harness is defined for both models
6. The evaluation metrics and outputs are stated clearly
7. A baseline Logistic Regression model is trained successfully
8. The baseline Logistic Regression model is evaluated on the validation set only
9. Results are presented clearly using the predefined evaluation harness
10. Markdown and code cells are organised clearly with appropriate headings

Quality criteria

1. Quality of data split and modelling setup

What is being judged: whether the train/validation/test setup is methodologically sound for this prediction task.

- 5 = The split is correct, clearly implemented, and methodologically sound, including appropriate handling of class imbalance such as stratification. The setup supports fair later comparison across models.
- 4 = The split is mostly sound and usable, with only minor weakness in implementation or explanation.
- 3 = The split is broadly correct but lacks rigour in one important aspect, such as weak handling of imbalance

or limited explanation.

- 2 = The split is simplistic or weakly implemented, with noticeable methodological concerns.
- 1 = The split is incorrect, poorly justified, or unsuitable for reliable model comparison.

2. Quality of preprocessing design

What is being judged: whether the preprocessing pipelines are appropriate for Logistic Regression, XGBoost, and the structure of the data.

- 5 = Preprocessing is well chosen for both models and clearly appropriate for the feature types and later modelling steps. It supports valid training without obvious unnecessary or unsuitable transformations.
- 4 = Preprocessing is mostly appropriate, with only minor weaknesses or inefficiencies.
- 3 = Preprocessing is broadly workable but somewhat generic, uneven, or only partly suited to the models or data.
- 2 = Preprocessing has clear weaknesses, questionable choices, or limited suitability for one or both models.
- 1 = Preprocessing is missing, clearly inappropriate, or likely to undermine valid modelling.

3. Quality of evaluation harness design

What is being judged: whether the evaluation metrics and outputs are well matched to the task, the data, and the intended comparison between models.

- 5 = The evaluation harness is well designed, clearly explained, and strongly aligned with the task context and data characteristics. Metric choice reflects the class imbalance and the substantive importance of identifying the minority class.
- 4 = The evaluation harness is solid and mostly well matched to the task, with only minor gaps in justification or metric selection.
- 3 = The evaluation harness is usable but somewhat generic, with limited attention to task context or imbalance.
- 2 = The evaluation harness is weakly justified or poorly matched to the task.
- 1 = The evaluation harness is missing, inappropriate, or not useful for fair comparison.

4. Quality of baseline Logistic Regression implementation

What is being judged: whether the baseline model is implemented in a clean, standard, and methodologically fair way.

- 5 = The baseline Logistic Regression is implemented cleanly and appropriately, with reasonable untuned hyperparameters, correct use of the validation set, and no sign of hidden tuning or misuse of the test set.
- 4 = The baseline implementation is mostly sound, with only minor weakness in clarity, parameter choice, or reporting.
- 3 = The baseline model is workable but somewhat generic, weakly explained, or methodologically uneven.
- 2 = The baseline implementation shows clear weaknesses, such as questionable setup, weak reporting, or limited fairness as a baseline.
- 1 = The baseline implementation is incorrect, poorly controlled, or not a valid baseline.

5. Quality of notebook communication and structure

What is being judged: whether the section is well organised and easy to follow as part of a reproducible modelling workflow.

- 5 = Markdown and code are very well structured, with clear headings, logical flow, and concise explanations of setup, evaluation logic, and results.
- 4 = The section is clear and readable overall, with only minor issues in organisation or explanation.
- 3 = The section is understandable but somewhat uneven in flow, explanation, or structure.
- 2 = The section is hard to follow in places because of weak organisation or unclear markdown.
- 1 = The section is poorly organised or confusing.

Step 5: Improving performance

Objective criteria (basic function)

1. The same training, validation, and test split from the previous section is reused
2. Logistic Regression is tuned using the validation set only
3. XGBoost is trained and tuned using the validation set only
4. No test-set information is used during tuning
5. A structured tuning summary is printed for tuned Logistic Regression
6. A structured tuning summary is printed for tuned XGBoost
7. Each tuning summary includes specified information
8. Test-set prediction is performed only after tuning is complete
9. Baseline Logistic Regression, tuned Logistic Regression, and XGBoost are all evaluated on the test set
10. Model comparison uses the predefined evaluation harness consistently
11. A model selection framework is defined in markdown
12. A final model is selected and justified using the stated framework
13. Markdown and code cells are organised clearly with appropriate headings

Quality criteria

1. Quality of tuning design and efficiency

What is being judged: whether tuning is methodologically sound, reasonably efficient, and clearly reported.

- 5 = Tuning is well designed, validation-only, and reasonably efficient. Search choices are sensible, clearly reported, and not unnecessarily wasteful. The structured tuning summary makes the process easy to audit and compare.
- 4 = Tuning is mostly sound and reasonably efficient, with only minor weakness in search design, reporting, or efficiency.
- 3 = Tuning is broadly workable but somewhat generic, inefficient, or only partly well reported.
- 2 = Tuning is weakly designed, poorly reported, or clearly inefficient relative to the task.
- 1 = Tuning is incorrect, poorly controlled, or not meaningfully interpretable.

2. Quality of XGBoost implementation and fairness of comparison

What is being judged: whether XGBoost is trained and compared in a fair and methodologically consistent way.

- 5 = XGBoost is implemented appropriately and compared fairly using the same split and evaluation framework as the other models. The comparison is methodologically clean and directly interpretable.
- 4 = XGBoost is mostly implemented and compared fairly, with only minor inconsistency or weakness.
- 3 = XGBoost is usable but the comparison is somewhat generic, uneven, or weakly controlled.
- 2 = XGBoost implementation or comparison has noticeable fairness or consistency issues.
- 1 = XGBoost is implemented incorrectly or compared in a way that is not reliable.

3. Quality of model comparison and test-set discipline

What is being judged: whether the final comparison is rigorous and respects the role of the test set.

- 5 = The test set is clearly reserved for final comparison only, and the three-model comparison is rigorous, consistent, and easy to interpret under the predefined evaluation harness.
- 4 = Test-set use is mostly disciplined and comparison is mostly clear, with only minor weakness.
- 3 = The comparison is broadly acceptable, but test-set discipline, consistency, or clarity is somewhat limited.
- 2 = The comparison is methodologically weak, unclear, or shows questionable handling of the test set.
- 1 = The comparison is unreliable because of misuse of the test set or inconsistent evaluation.

4. Quality of final model selection framework

What is being judged: whether the final model is chosen using a clear, task-appropriate, and genuinely multi-

dimensional decision framework.

- 5 = The framework is quantitative, uses reasonable scoring or weighting, prioritises metrics appropriately for this task, and considers more than predictive performance, such as interpretability or practical usability.
- 4 = The framework is clear and mostly well designed, with sensible criteria and some multi-dimensional thinking, but one element is underdeveloped.
- 3 = The framework is usable but fairly generic, with limited quantification or limited consideration beyond model performance.
- 2 = The framework is weakly defined, relies mostly on raw performance comparison, or shows poor prioritisation of criteria for the task.
- 1 = The framework is missing, unclear, or not meaningfully used to choose the final model.

5. Quality of notebook communication and structure

What is being judged: whether the section is well organised and easy to follow as part of a reproducible model-improvement workflow.

- 5 = Markdown and code are very well structured, with clear headings, logical flow, and concise explanations of tuning, comparison, and final decision.
- 4 = The section is clear and readable overall, with only minor issues in organisation or explanation.
- 3 = The section is understandable but somewhat uneven in flow, explanation, or structure.
- 2 = The section is hard to follow in places because of weak organisation or unclear markdown.
- 1 = The section is poorly organised or confusing.

Step 6: Producing reproducible packaging

Objective criteria (basic function)

1. No new content is added to the notebook in this step
2. A requirements.txt file is created in the working directory
3. requirements.txt includes the Python packages used in the experiment
4. A README.md file is created in the working directory
5. The README states the required input files
6. The README explains how to run the notebook
7. The README describes the main outputs and where they are saved
8. The README states the main reproducibility measures used

Quality criteria

1. Quality of requirements.txt

What is being judged: whether the dependency file is complete, correct, and reasonably minimal.

- 5 = requirements.txt is complete, correct, and reasonably minimal. It includes the packages actually used, without obvious important omissions or unnecessary clutter.
- 4 = requirements.txt is mostly complete and usable, with only minor omission, redundancy, or imprecision.
- 3 = requirements.txt is broadly usable but somewhat incomplete, overly broad, or loosely matched to the actual notebook.
- 2 = requirements.txt has noticeable omissions or unnecessary entries that reduce reproducibility or clarity.
- 1 = requirements.txt is missing, seriously incomplete, or not usable.

2. Quality of README.md for reproducibility

What is being judged: whether the README is clear enough for an unfamiliar team to rerun the experiment.

- 5 = The README is clear, concise, and practically useful for a new team. It explains required files, execution steps, outputs, save locations, and reproducibility measures in a way that would realistically support rerunning the workflow.

- 4 = The README is mostly clear and usable, with only minor gaps or ambiguity.
- 3 = The README is broadly understandable but somewhat incomplete, generic, or only partly sufficient for reliable rerunning.
- 2 = The README is vague or incomplete, and would make rerunning difficult for an unfamiliar team.
- 1 = The README is missing, very unclear, or not useful for reproducibility.

3. Quality of consistency with notebook outputs

What is being judged: whether the packaging matches what was actually done and produced in the notebook.

- 5 = The packaging is fully consistent with the notebook workflow, methods, and outputs. File names, output locations, and reproducibility notes align clearly with the implemented experiment.
- 4 = The packaging is mostly consistent with the notebook, with only minor mismatch or omission.
- 3 = The packaging is broadly related to the notebook, but some details are generic, incomplete, or not fully aligned.
- 2 = The packaging shows noticeable mismatch with the notebook workflow or outputs.
- 1 = The packaging is seriously inconsistent with the notebook or appears largely disconnected from the actual experiment.

4. Quality of communication and packaging clarity

What is being judged: whether the packaging is presented in a clean, professional, and easy-to-audit way.

- 5 = The packaging is concise, well organised, and easy to inspect. It communicates the experiment setup clearly without unnecessary detail.
- 4 = The packaging is clear overall, with only minor issues in clarity or organisation.
- 3 = The packaging is understandable but somewhat uneven, wordy, or lacking polish.
- 2 = The packaging is hard to follow in places because of weak organisation or unclear wording.
- 1 = The packaging is confusing, poorly organised, or difficult to use.

Step 7: Writing documentation

Objective criteria (basic function)

1. No new content is added to the notebook in this step
2. A file named Report_[agent_name].md is created in the workspace
3. The report is written in English and markdown format
4. The report is approximately 400 words
5. The report is written for a non-technical government arts audience
6. The report covers specified sections
7. The report uses the actual notebook results rather than placeholder language

Quality criteria

1. Quality of grounding in notebook results

What is being judged: whether the report accurately reflects the actual notebook outputs and does not invent unsupported claims.

- 5 = The report is clearly grounded in the notebook results, accurately reflects the model comparison and final choice, and does not introduce unsupported claims or made-up details.
- 4 = The report is mostly well grounded in the notebook results, with only minor imprecision or slight overgeneralisation.
- 3 = The report is broadly consistent with the notebook, but some statements are generic, weakly supported, or not tightly tied to the actual results.
- 2 = The report only partly reflects the notebook results and includes noticeable vagueness, mismatch, or unsupported interpretation.

- 1 = The report is largely ungrounded, misleading, or inconsistent with the notebook outputs.

2. Quality of communication for a non-technical policy audience

What is being judged: whether the report is genuinely understandable and appropriate for a government arts audience.

- 5 = The report is clear, accessible, and well pitched for a non-technical policy audience. It explains the work without unnecessary jargon and keeps the focus on what matters for decision-making.
- 4 = The report is mostly clear and suitable for the intended audience, with only minor issues in tone or accessibility.
- 3 = The report is understandable overall, but somewhat generic, uneven, or too technical in places.
- 2 = The report is only partly suitable for a non-technical audience and contains substantial jargon, weak framing, or unclear explanation.
- 1 = The report is poorly matched to the audience and difficult for non-technical readers to use.

3. Quality of rationale and practical interpretation

What is being judged: whether the report explains the model choice and practical meaning of the results in a sensible and convincing way.

- 5 = The report gives a clear and credible rationale for the final model choice, links it to the purpose of the task, and draws practical implications that are relevant and well reasoned.
- 4 = The rationale and interpretation are mostly clear and sensible, with only minor weakness in depth or connection to the task.
- 3 = The rationale is broadly acceptable, but somewhat generic, thin, or only partly convincing.
- 2 = The rationale is weak, unclear, or only loosely connected to the task and results.
- 1 = The rationale is missing, implausible, or not meaningfully supported.

4. Quality of caution and responsible reporting

What is being judged: whether the report communicates limitations appropriately and avoids overclaiming.

- 5 = The report communicates limitations clearly, avoids causal overclaiming, and presents the model as decision support rather than definitive truth.
- 4 = The report is mostly responsible and appropriately cautious, with only minor overstatement or incomplete caveats.
- 3 = The report includes some caution, but limitations are underdeveloped or the tone is somewhat overconfident.
- 2 = The report gives weak or minimal caution and risks overstating what the model can support.
- 1 = The report is clearly overclaimed, misleading, or lacks meaningful limitations.

Appendix C: Agent Run Logs

Each agent maintained a run log recording step-by-step progress, completion status, key actions, outputs, and any warnings or errors. These logs are reproduced below.

C1 Claude Code

Run Log — claude_code

Generated: 2026-03-17 02:06:24

Step: 0_setup

- Completion Status: SUCCESS
- Key Actions: Created notebook, evidence directory, run log
- Key Outputs: experiment_claude_code.ipynb, evidence_claude_code/, run_log_claude_code.md
- Warnings/Errors: None

Step: 1.1_dataset_ingestion

- Completion Status: SUCCESS
- Key Actions: Loaded 34378 rows x 16 cols; verified all 16 variables present
- Key Outputs: participation_raw dataframe
- Warnings/Errors: 88 duplicate rows found

Step: 2_EDA_prep

- Completion Status: SUCCESS
- Key Actions: Removed 40 rows with CARTS_NET in [-3,3]; created binary target
- Key Outputs: participation_eda dataframe
- Warnings/Errors: None

Step: 2_EDA

- Completion Status: SUCCESS
- Key Actions: Generated target distribution, feature distributions, engagement-rate plots, correlation heatmap
- Key Outputs: EDA_claude_code_Pics/: target_distribution.png, feature_distributions.png, feature_target_relationships.png, correlation_heatmap.png
- Warnings/Errors: None

Step: 3_missingness_handling

- Completion Status: SUCCESS
- Key Actions: Recoded CULTSATIS -3→6; dropped rows with non-informative codes; 5265

rows removed (15.3%)

- Key Outputs: participation_clean dataframe, evidence_claude_code/missingness_handling_summary.csv
- Warnings/Errors: None

Step: 4.1_prepare_modeling_data

- Completion Status: SUCCESS
- Key Actions: Defined X (15 features) and y; created LR and XGB preprocessors; stratified 70/15/15 split
- Key Outputs: X_train, X_val, X_test, y_train, y_val, y_test; preprocessed matrices
- Warnings/Errors: None

Step: 4.2_evaluation_harness

- Completion Status: SUCCESS
- Key Actions: Defined evaluate_model() with Precision, Recall, F1, F2, Balanced Accuracy, ROC-AUC, PR-AUC
- Key Outputs: evaluate_model() function
- Warnings/Errors: None

Step: 4.3_baseline_LR

- Completion Status: SUCCESS
- Key Actions: Trained baseline LR (C=1.0, default threshold 0.5)
- Key Outputs: baseline_lr model, evidence_claude_code/baseline_lr_validation_metrics.csv
- Warnings/Errors: None

Step: 5.1_tune_LR

- Completion Status: SUCCESS
- Key Actions: Grid search over 9 C values x 17 thresholds = 153 configs; best C=10.0, threshold=0.55
- Key Outputs: tuned_lr model, evidence_claude_code/lr_tuning_results.csv
- Warnings/Errors: None

Step: 5.2_tune_XGBoost

- Completion Status: SUCCESS
- Key Actions: Grid search over 27 configs x 17 thresholds = 459 evaluations; best: n_est=300, max_depth=7, lr=0.1, threshold=0.9
- Key Outputs: tuned_xgb model, evidence_claude_code/xgb_tuning_results.csv
- Warnings/Errors: None

Step: 5.3_model_comparison

- Completion Status: SUCCESS
- Key Actions: Evaluated all 3 models on held-out test set
- Key Outputs: evidence_claude_code/test_model_comparison.csv
- Warnings/Errors: None

Step: 5.4_final_model_decision

- Completion Status: SUCCESS
- Key Actions: Applied weighted multi-criteria framework; selected Tuned LR (score=0.7169)
- Key Outputs: evidence_claude_code/model_selection_framework.csv, best_model_name.txt
- Warnings/Errors: None

C2 Codex

Run Log: codex

Step 0 Setup

- completion status: Completed
- key actions:
  - Reviewed variable dictionary only (no data load in this step).
  - Created notebook `experiment_codex.ipynb` with title and setup section.
  - Set reproducibility controls with fixed seed `42`.
  - Created `evidence_codex/` and EDA figure subfolder prepared by notebook.
- key outputs:
  - `experiment_codex.ipynb`
  - `evidence_codex/`
- important warnings or errors:
  - None.

Step 1.1 Dataset Ingestion and Schema Checks

- completion status: Completed
- key actions:
  - Loaded `data/participation_2024-25_experiment_v2.tab` into `participation_raw`.
  - Checked required variable presence and obvious schema issues.
- key outputs:
  - Raw shape confirmed in notebook: `34,378 rows x 16 columns`.
  - All expected required variables present; no null-based schema break.
- important warnings or errors:
  - None.

Step 1.2 Problem Definition

- completion status: Completed
- key actions:
  - Added task framing markdown for policy-focused under-engagement identification.
  - Declared target and features.
  - Explicitly stated `CARTS_NET` values `-3` and `3` will be dropped later.
  - Added variable table based on dictionary.
- key outputs:
  - Problem definition markdown section in notebook.
- important warnings or errors:
  - None.

Step 2 EDA

- completion status: Completed
- key actions:
  - Dropped target-coded missing rows (`CARTS_NET` in `{-3, 3}`).
  - Built `participation_eda` by removing original `CARTS_NET` and adding binary-labelled target.
  - Produced EDA visualisations and saved PNG outputs.
- key outputs:
  - EDA sample rows after target filtering: `34,338`.
  - `evidence_codex/EDA_codex_Pics/01_target_distribution.png`
  - `evidence_codex/EDA_codex_Pics/02_ageband_target_stacked.png`
  - `evidence_codex/EDA_codex_Pics/03_education_target_stacked.png`
  - `evidence_codex/EDA_codex_Pics/04_deprivation_underengagement_line.png`
  - `evidence_codex/EDA_codex_Pics/05_wellb1_kde_by_target.png`
  - `evidence_codex/EDA_codex_Pics/06_cultsatis_distribution.png`
- important warnings or errors:
  - None.

Step 3 Missingness Handling

- completion status: Completed
- key actions:
  - Applied variable-specific rules from coded meanings and missing rates.
  - Dropped low-rate (<5%) non-informative coded responses.
  - Re-coded high-rate coded missing values to `Unknown` for selected variables.
  - Treated `CULTSATIS=-3` as informative `RoutingSkip`.
  - Kept `EDUCAT3=-3` as informative `NoQualifications`.
  - Produced cleaned dataset `participation_clean` with binary numeric target.
- key outputs:
  - `evidence_codex/missingness_profile.csv`
  - `evidence_codex/participation_clean.csv`
  - Rows before cleaning: `34,338`
  - Rows after cleaning: `28,995`
- important warnings or errors:
  - None.

Step 4.1 Prepare Modelling Data

- completion status: Completed

- key actions:
  - Defined `X` and `y` from `participation_clean`.
  - Built preprocessing pipelines for Logistic Regression and XGBoost.
  - Created fixed stratified train/validation/test splits (0.70/0.15/0.15).
- key outputs:
  - Split created and reused for all later tuning/evaluation cells.
- important warnings or errors:
  - None.

Step 4.2 Create Evaluation Harness

- completion status: Completed
- key actions:
  - Defined unified evaluation metrics and threshold-search utility.
  - Prioritised F2 and recall for under-engagement detection use case.
- key outputs:
  - Reusable evaluation functions in notebook.
- important warnings or errors:
  - None.

Step 4.3 Baseline Logistic Regression

- completion status: Completed
- key actions:
  - Trained baseline LR on train set.
  - Evaluated on validation set only with common harness.
- key outputs:
  - Baseline model metrics recorded in notebook.
- important warnings or errors:
  - None.

Step 5.1 Improve Logistic Regression

- completion status: Completed
- key actions:
  - Tuned LR hyperparameters on validation set only.
  - Performed threshold optimisation for F2.
- key outputs:
  - `evidence_codex/lr_tuning_summary.json`
  - Best LR validation setup: `C=0.05`, `penalty=l1`, `class_weight=balanced`, `threshold=0.52`.

- important warnings or errors:
  - None.

Step 5.2 Tune XGBoost

- completion status: Completed
- key actions:
  - Trained and tuned XGBoost on the same fixed split.
  - Performed threshold optimisation for F2.
- key outputs:
  - `evidence_codex/xgb_tuning_summary.json`
  - Best XGBoost validation setup: `n_estimators=200`, `max_depth=4`, `learning_rate=0.08`, `threshold=0.10`.
- important warnings or errors:
  - None.

Step 5.3 Model Comparison (Test Set Only)

- completion status: Completed
- key actions:
  - Used test set only at final comparison stage.
  - Compared baseline LR, tuned LR, tuned XGBoost under identical harness.
- key outputs:
  - `evidence_codex/model_comparison_test.csv`
  - Test F2: tuned LR `0.4639`, tuned XGBoost `0.4515`, baseline LR `0.0721`.
- important warnings or errors:
  - None.

Step 5.4 Final Model Decision

- completion status: Completed
- key actions:
  - Defined quantitative multi-metric model selection score.
  - Selected final model based on weighted policy-oriented criteria.
- key outputs:
  - `evidence_codex/final_model_selection_score.csv`
  - Final model selected: `tuned_lr` (selection score `0.4875`).
- important warnings or errors:
  - None.

Step 6 Producing Reproducible Packaging

- completion status: Completed
- key actions:
  - Created concise dependency file and run instructions.
- key outputs:
  - `requirements.txt`
  - `README.md`
- important warnings or errors:
  - None.

Step 7 Writing Documentation

- completion status: Completed
- key actions:
  - Produced policy-facing non-technical markdown report using actual model outputs.
- key outputs:
  - `Report_codex.md`
- important warnings or errors:
  - None.

Notes

- The notebook initially failed execution due malformed indentation in generated cells; cells were corrected and notebook was rerun successfully end-to-end.

C3 Antigravity

Run Log: antigravity

0_setup

- **Status:** Completed
- **Key Actions:** Imported libraries, set random seed to 42, created tracking folders.
- **Key Outputs:** evidence folder, run log file.
- **Warnings/Errors:** None

1.1_dataset_ingestion_and_schema_checks

- **Status:** Completed
- **Key Actions:** Loaded raw data and verified presence of required variables.
- **Key Outputs:** Raw data shape: (34378, 16)
- **Warnings/Errors:** None

1.2_problem_defition

- **Status:** Completed
- **Key Actions:** Defined prediction task and variables in markdown.
- **Key Outputs:** None
- **Warnings/Errors:** None

2_EDA

- **Status:** Completed
- **Key Actions:** Created binary target, dropped missing targets. Generated and saved EDA plots.
- **Key Outputs:** EDA plots in EDA_antigravity_Pics folder.
- **Warnings/Errors:** None

3_missingness_handling

- **Status:** Completed
- **Key Actions:** Dropped rows with non-informative missing codes according to rules.
- **Key Outputs:** Cleaned data shape: (24867, 16)
- **Warnings/Errors:** None

4.1_prepare_modeling_data

- **Status:** Completed
- **Key Actions:** Split data into train, val, test and built preprocessor.
- **Key Outputs:** Train/Val/Test splits.
- **Warnings/Errors:** None

4.2_create_evaluation_harness

- **Status:** Completed
- **Key Actions:** Defined evaluation metrics focusing on F2 Score and PR-AUC.
- **Key Outputs:** evaluate_model function.
- **Warnings/Errors:** None

4.3_baseline_model_LR

- **Status:** Completed
- **Key Actions:** Trained baseline Logistic Regression model.
- **Key Outputs:** Baseline validation metrics.
- **Warnings/Errors:** None

5.1_improve_LR

- **Status:** Completed
- **Key Actions:** Tuned LR and found best threshold on validation set.
- **Key Outputs:** Tuned LR settings and validation metrics.
- **Warnings/Errors:** None

5.2_tune_XGBoost

- **Status:** Completed
- **Key Actions:** Tuned XGBoost and found best threshold on validation set.
- **Key Outputs:** Tuned XGB settings and validation metrics.
- **Warnings/Errors:** None

5.3_model_comparison

- **Status:** Completed
- **Key Actions:** Evaluated all three models on the test set.
- **Key Outputs:** Test set metrics.
- **Warnings/Errors:** None

5.4_final_model_decision

- **Status:** Completed
- **Key Actions:** Compared test metrics and selected final model.
- **Key Outputs:** Saved final_model.pkl to evidence.
- **Warnings/Errors:** None

Appendix D: Member contribution

Name	Contribution
Chase Sun	Experiments, Comparative analysis, Repo management
Clara Yang	Literature review
Leah Luo	Comparative analysis
Xiaocheng Wang	Benchmark design